學完 CSS Selector 後，接下來要學習 XPath。CSS Selector 像是透過「長相（Class、ID）」找人，而 XPath 則像是透過「地址（路徑）」找人。在處理複雜 HTML 或 XML 時，XPath 往往更靈活且功能更強大。

9-1 XPath 語法：絕對路徑與相對路徑

一、觀念介紹

1. 什麼是 XPath？

- 用來在 XML 或 HTML 文件中尋找節點的語言。
- 不只能往下找子節點，也能往上找父節點，彈性比 CSS Selector 更高。

2. 絕對路徑 (Absolute Path)

- 以 / 開頭。
- 從根節點一路往下撰寫完整路徑。
- 缺點是網頁結構稍有變動就可能失效。

3. 相對路徑 (Relative Path)

- 以 // 開頭。
- 不需理會前面層級，可直接搜尋目標節點。
- 穩定且簡潔，是實務開發最常用的方式。

二、程式範例

範例 1. 使用 lxml 解析 HTML 並執行基礎 XPath

```
from lxml import etree
html_str = """
<html>
    <body>
        <div class="content">
            <h1 id="title">XPath 學習</h1>
            <p>第一段文字</p>
            <p>第二段文字</p>
        </div>
    </body>
</html>
"""
# 將字串轉換為 HTML 元素物件
content = etree.HTML(html_str)
```

```
# 1. 使用絕對路徑抓取 h1 (不建議實戰使用)
title_abs = content.xpath("/html/body/div/h1")
print(f"絕對路徑結果：{title_abs[0].text}")
# 2. 使用相對路徑抓取 h1 (推薦！)
title_rel = content.xpath("//h1")
print(f"相對路徑結果：{title_rel[0].text}")
```

範例 2. 局部路徑查找（.//）

```
from lxml import etree
html = """
<div class="user-card">
    <span class="name">老王</span>
</div>
<div class="user-card">
    <span class="name">小李</span>
</div>
"""
tree = etree.HTML(html)
cards = tree.xpath("//div[@class='user-card']")
for card in cards:
    # 注意這裡的 .// 代表「從當前節點開始找」
    # 若沒加點，會變成從整份文件的開頭找
    name = card.xpath(".//span/text()")
    print(f"使用者：{name[0]}")
```

範例 3. 多層級的靈活跳轉

```
from lxml import etree
html = "<div><section><article><ul><li>目標項目
</li></ul></article></section></div>"
tree = etree.HTML(html)

# 不管中間有多少層，直接找 div 下面的 li
# // 代表中間可以隔著任何標籤
target = tree.xpath("//div//li/text()")
print(f"抓到的文字：{target[0]}")
```

三、程式說明

1. etree.HTML(html_str)

- 將 HTML 轉換成可查詢的樹狀結構。
- 能自動補齊部分不完整的 HTML 標籤。

2. .xpath( ) 的回傳值

- 永遠回傳 List（清單）。
- 即使只找到一個元素，也要透過 [0] 取出。

3. // 與 / 的差異

- //：可跨越任意層級搜尋。
- /：只能搜尋直接下一層。
- 兩者可混合使用提升精準度。

四、常見錯誤

錯誤 1：忘記加點點（.）

- 錯誤：item.xpath("//span")
- 結果：會從整份文件重新搜尋。
- 正確：item.xpath(".//span")

錯誤 2：索引觀念混淆

- XPath 選取位置從 1 開始。
- Python List 索引從 0 開始。

錯誤 3：路徑寫太死

- Copy Full XPath 通常非常冗長，網頁稍微修改就會失效。
- 建議優先使用相對路徑。

五、延伸重要觀念

1. lxml 的效能優勢

- 採用 C 語言實作。
- 處理大型 HTML 時速度通常比 BeautifulSoup 更快。

2. 瀏覽器工具輔助

- Chrome 開發者工具（F12）可直接測試 XPath。
- 使用 Ctrl + F 輸入 XPath 即可驗證結果。

3. 與 CSS Selector 的對應關係

XPath	CSS Selector
//div	div
//div/p	div > p
//div//p	div p

9-1 XPath 語法：絕對路徑與相對路徑 隨堂練習

一、選擇題 (共 10 題)

1. 關於 XPath 的敘述，下列何者「錯誤」？
   (A) 它是一種用來在 XML 或 HTML 文件中尋找節點的語言
   (B) 與 CSS Selector 相比，XPath 具有可以向上尋找父節點的靈活性
   (C) XPath 的執行速度很慢，因此實戰中完全不被推薦使用
   (D) XPath 可以透過「地址（路徑）」的概念來精準定位網頁元素
2. 在 XPath 語法中，以單斜槓 / 開頭的路徑稱為什麼路徑？
   (A) 絕對路徑 (B) 相對路徑 (C) 局部路徑 (D) 全域路徑
3. 下列關於「絕對路徑」的缺點描述，何者正確？
   (A) 絕對路徑的搜尋速度比相對路徑慢上數萬倍
   (B) 結構極其脆弱，網頁稍微多一個 div 或改動位置，路徑就會失效
   (C) 絕對路徑無法在 Python 程式中被正確執行
   (D) 絕對路徑會強迫瀏覽器重新整理網頁
4. 在 XPath 語法中，若想要在文件的任何地方開始搜尋，不需要管它前面的層級，應該使用什麼符號開頭？
   (A) 單斜槓 / (B) 雙斜槓 // (C) 點斜槓 ./ (D) 點雙斜槓 .//
5. 使用 lxml 套件的 etree.HTML(html_str) 方法時，它具備什麼特點？
   (A) 會自動下載該網頁的圖片資料
   (B) 會自動補全不完整的 HTML 標籤並建立樹狀結構
   (C) 會直接將 HTML 轉成 JSON 格式
   (D) 會強制刪除網頁中的所有 JavaScript 程式碼
6. Python 呼叫元素物件的 .xpath() 方法後，其回傳的資料型態永遠為何？
   (A) 字典 (Dictionary) (B) 字串 (String) (C) 元素物件本身 (Tag) (D) 列表 (List)
7. 在進行容器化局部查找的迴圈中，若想從「目前當前節點」開始向下尋找 span 標籤，正確的 XPath 寫法為何？
   (A) //span (B) /span (C) .//span (D) ..//span
8. 小明在寫 XPath 迴圈時，忘記在局部路徑前加上點點（.），直接寫了 item.xpath("//span")。請問會發生什麼慘劇？
   (A) 程式會直接噴出錯誤並中斷執行
   (B) 會重新從「整份網頁」開頭找 span，導致每一圈抓到的都是全網頁的第一個 span
   (C) 什麼東西都抓不到，一律回傳空的 List
   (D) 會導致 lxml 無法解析網頁而當掉
9. 在 XPath 的原生語法中，若要選取第一個 <li> 標籤，其索引值（Index）是從多少開始？
   (A) 1 (B) 0 (C) -1 (D) 沒有索引值的概念

10. 關於 XPath 與 CSS Selector 的對應關係，下列哪一個選項是正確的？

(A) XPath 的 //div/p 等同於 CSS 的 div p

(B) XPath 的 //div//p 等同於 CSS 的 div > p

(C) XPath 的 //div 等同於 CSS 的 div

(D) XPath 的 /html/body 等同於 CSS 的 html body

二、填充題 (共 5 題)

1. 在 lxml 套件中，將 HTML 字串轉換為可供查詢的樹狀結構核心方法是 etree._____()。
2. 絕對路徑必須從根節點（通常是 _____）開始一層一層寫下來。
3. 如果想在全網頁找所有的 ，然後找它「直接下一層」的 <p>，其混用的 XPath 語法應該寫成 _____。
4. 由於 lxml 是用 _____ 語言編寫的底層套件，因此在處理超大型網頁時，速度比傳統解析套件快非常多。
5. 在 Chrome 瀏覽器的 F12 開發者工具面板中，可以按下快速鍵 _____ 來開啟搜尋框，即時測試 XPath 語法。

三、實作題 (共 2 題)

實作題一：成員卡片局部路徑提取

題目說明： 請撰寫一段 Python 程式碼，使用 lxml 套件解析含有多個成員卡片的 HTML 字串。

1. 使用相對路徑先找出所有的成員容器（具有 class='member-card' 的 div）。
2. 使用 for 迴圈遍歷每個容器。
3. 在迴圈內部，務必使用局部相對路徑（帶有點點）分別提取成員的姓名（class='name'）與職稱（class='title'）的純文字內容。
4. 將每位成員的資料格式化印出。

提示：

- 使用 tree.xpath("//div[@class='member-card']") 抓取容器。
- 迴圈內部使用 card.xpath(".//span[@class='name']/text()") 提取文字，並記得使用 [0] 取出列表中的字串。

輸入範例一

```
html_str = """
<div class="directory">
    <div class="member-card">
        <span class="name">張經理</span>
        <span class="title">專案主管</span>
    </div>
    <div class="member-card">
        <span class="name">李工程師</span>
        <span class="title">後端開發</span>
    </div>
</div>
"""
```

輸出結果一

```
成員：張經理，職稱：專案主管
成員：李工程師，職稱：後端開發
```

輸入範例二

```
html_str = """
<div class="directory">
    <div class="member-card">
        <span class="name">王總監</span>
        <span class="title">技術指導</span>
    </div>
</div>
"""
```

輸出結果二

```
成員：王總監，職稱：技術指導
```

實作題二：跨層級靈活跳轉提取

題目說明：有時網頁結構階層非常深，如果寫絕對路徑容易失效。請利用 XPath 雙斜槓 // 的靈活跳轉特性，撰寫一段程式碼。

1. 解析指定的 HTML 字串。
2. 鎖定具有 id='main-content' 的 div 標籤。
3. 不管中間隔了多少層標籤（例如 section, article, ul），直接獲取該 div 內部所有 <li> 標籤裡面的純文字。
4. 使用迴圈將所有抓到的項目文字依序印出。

提示：

- 定位路徑可以寫成 //div[@id='main-content']//li/text()。
- 這樣一來，中間層級無論如何變動，只要 div 裡還有 li 就能成功抓取。

輸入範例一

```
html_str = """
<div id="main-content">
    <section>
        <article>
            <ul>
                <li>Python 爬蟲特訓</li>
                <li>自動化測試實戰</li>
            </ul>
        </article>
    </section>
</div>
"""
```

輸出結果一

```
課程項目：Python 爬蟲特訓
課程項目：自動化測試實戰
```

輸入範例二

```
html_str = """
<div id="main-content">
    <div>
        <ol>
            <li>Java 基礎教學</li>
        </ol>
    </div>
</div>
"""
```

輸出結果二

```
課程項目：Java 基礎教學
```

9-2 屬性與文字選取：@attr 與 text( )

XPath 可以直接在路徑中指定要取得「文字」或「屬性」，不需要先抓到標籤再使用 .text 或 .get( )，因此程式通常更精簡。

一、觀念介紹

1. 選取文字：text( )

- 用來取得節點中的文字內容。
- 不包含子標籤內的文字。

2. 選取屬性：@屬性名

- 可直接取得 href、src、title 等屬性值。
- 常用於抓取連結與圖片網址。

3. 回傳結果變化

- xpath("//div") → 標籤物件清單。
- xpath("//div/text( )") → 字串清單。
- xpath("//a/@href") → 屬性值清單。

二、程式範例

範例 1. 提取純文字與清理

```
from lxml import etree
html_str = """
<div class="header">
    <h1> 歡迎來到 Python 爬蟲教室 </h1>
    <p>這是一堂關於 <b>XPath</b> 的課程</p>
</div>
"""
tree = etree.HTML(html_str)
# 1. 提取 h1 的文字 (包含空白)
raw_text_list = tree.xpath("//h1/text( )")
raw_text = raw_text_list[0] if raw_text_list else ""
print(f"原始文字：'{raw_text}'")
# 2. 結合 Python 進行清理
clean_text = raw_text.strip( )
print(f"清理後：'{clean_text}'")
# 3. 提取 p 的文字 (注意：這不會包含 b 標籤裡的 'XPath')
p_text = tree.xpath("//p/text( )")
print(f"P 標籤文字清單：{p_text}")
```

範例 2. 提取多個屬性值 (連結與圖片)

```
from lxml import etree
html = """
<div class="gallery">
    <a href="https://example.com/1">
        <img src="thumb1.jpg" alt="圖片 1">
    </a>
    <a href="https://example.com/2">
        <img src="thumb2.jpg" alt="圖片 2">
    </a>
</div>
"""
tree = etree.HTML(html)
# 1. 抓取所有連結
links = tree.xpath("//a/@href")
print(f"所有連結：{links}")
# 2. 抓取所有圖片的替代文字 (alt)
alts = tree.xpath("//img/@alt")
print(f"圖片描述：{alts}")
```

範例 3. 同時選取多個節點的文字 XPath

```
from lxml import etree
html = """
<ul>
    <li id="id_01">項目一</li>
    <li id="id_02">項目二</li>
    <li id="id_03">項目三</li>
</ul>
"""
tree = etree.HTML(html)
# 抓取所有 li 的 id 屬性與文字
ids = tree.xpath("//li/@id")
texts = tree.xpath("//li/text()")
# 使用 zip 縫合資料
for i, t in zip(ids, texts):
    print(f"編號：{i} | 內容：{t}")
```

三、程式說明

1. text( ) 的局限性

- 只抓取目前節點的文字。
- 不會包含子標籤內的內容。

2. @ 符號的用途

- 可取得任何 HTML 屬性。
- 包含自訂屬性 (如 data-price)。

3. 回傳值處理

- 找不到資料時回傳空清單 []。
- 使用前建議先判斷是否有內容。

四、常見錯誤

1. 錯誤 1：對字串使用 .text

- xpath("//h1/text( )") 取得的是字串。
- 字串物件沒有 .text 屬性。

2. 錯誤 2：忽略多個文字節點

- 可能取得多個文字片段。
- 可利用 "".join(list).strip( ) 合併與清理。

3. 錯誤 3：屬性名稱大小寫錯誤

- XPath 區分大小寫。
- @href 與 @HREF 是不同屬性。

五、延伸重要觀念

1. string(.) 與 text( ) 的差異

- **text( )**：只抓目前層文字。
- **string( )**：抓取包含所有子標籤的完整文字。
- **範例**：tree.xpath("string(//div[@class='header'])")

2. 選取所有屬性

- 使用 //@* 可取得所有屬性值。

3. 性能優勢

- XPath 可直接由底層完成資料提取。
- 大量節點處理時通常比逐一讀取標籤更有效率。

9-2 屬性與文字選取：@attr 與 text( ) 隨堂練習

一、選擇題 (共 10 題)

1. 在 XPath 語法中，若想要直接在路徑最後提取某個節點下的「純文字內容」，應該使用哪一個函數？ (A) @text (B) text( ) (C) string( ) (D) .text
2. 在 XPath 語法中，若要直接提取一個 <a> 標籤內部的 href 屬性值，正確的路徑結尾寫法為何？ (A) //a/href (B) //a(href) (C) //a/@href (D) //a/.href
3. 執行 tree.xpath("//a/@href") 後，回傳給 Python 的資料型態是什麼？
   (A) 標籤物件列表 (List of Tags) (B) 字串列表 (List of Strings)
   (C) 單一字串 (String) (D) 字典列表 (List of Dictionaries)
4. 若網頁結構為 ABC ，當執行 tree.xpath("//div/text( )") 時，回傳的清單內容為何？ (A) ['A', 'B', 'C'] (B) ['A B C'] (C) ['A', 'C'] (D) ['B']
5. 關於 XPath 提取屬性的描述，下列何者「錯誤」？
   (A) XPath 只能提取 href 或 src，無法提取自定義屬性（如 data-price）
   (B) 使用 @* 可以選取某個標籤內部的所有屬性值
   (C) XPath 對大小寫極其敏感，@href 與 @HREF 代表完全不同的屬性
   (D) 直接在路徑中提取屬性，其執行效率通常高於抓到標籤後再用 Python 迴圈提取
6. 小明執行了 results = tree.xpath("//h1/text( )") 成功拿到資料，接著他寫了 print(results[0].text) 卻遇到程式報錯，請問原因為何？
   (A) results[0] 已經是 Python 字串物件，字串沒有 .text 屬性
   (B) h1 標籤內部不能包含任何文字
   (C) 必須寫成 results[0].text( ) 才能執行
   (D) xpath( ) 方法回傳的是字典，不能用索引值 [0] 取值
7. 當某個網頁標籤內部穿插了許多換行符號，導致 text( ) 回傳了 ['\n ', '實際文字', '\n']，下列哪一種 Python 寫法最適合用來將它們合併並清理乾淨？
   (A) "".join(list).strip( ) (B) list.replace("\n", "") (C) list[0].strip( ) (D) list.strip( )
8. 如果想要抓取某個標籤內「包含所有子標籤內部文字」的完整串接內容（類似 BeautifulSoup 的 .text），在 XPath 中應改用什麼函數？
   (A) all_text( ) (B) text(all) (C) string( ) (D) get_text( )
9. 當 tree.xpath( ) 在網頁中完全找不到任何匹配的標籤、文字或屬性時，會回傳什麼內容給 Python？ (A) None (B) 空列表 [] (C) 拋出 AttributeError 異常 (D) 空字串 ""
10. 關於 XPath 性能優勢的敘述，下列何者正確？
    (A) 因為 XPath 是用 Java 語言在底層運作，所以速度特別快
    (B) 將資料提取邏輯直接寫在路徑中，交由底層的 C 語言（lxml）處理，效率極高
    (C) 只有在搭配 CSS Selector 一起混寫時才能提升速度
    (D) 它的主要優勢在於能自動過濾所有網路廣告

二、填充題 (共 5 題)

1. 在 XPath 中，選取屬性必須使用 _____ 符號後接屬性名稱。
2. 函數 text( ) 只會抓取「_____層級」的文字節點，而不會包含子標籤內部的文字。
3. 若網頁當中有一個自定義屬性標籤如 ，要直接提取該價格，XPath 可以寫為 //@_____。
4. 為了將多個 <li> 標籤同時提取出來的 ids 列表與 texts 列表一一對應縫合，可以利用 Python 內建的 _____ 函數。
5. 想要獲取一個標籤到底有哪些屬性，可以使用萬用屬性選擇語法 //@_____。

三、實作題 (共 2 題)

實作題一：電商圖片與連結清單一條龍提取

題目說明： 請撰寫一段 Python 程式碼，使用 lxml 套件解析商品展示牆的 HTML 字串。

1. 請直接編寫一條 XPath 路徑，提取出所有商品 <a> 標籤的 href 連結字串列表。
2. 請再編寫一條 XPath 路徑，提取出所有 <img> 標籤的 src 圖片網址字串列表。
3. 使用 zip( ) 函數將這兩個字串列表組合，並依序印出。

提示：

- 提取連結路徑：//a/@href
- 提取圖片路徑：//img/@src
- 不需要使用 for 迴圈抓取容器後再 .xpath( )，本題要求直接從全域提取出字串列表後再行縫合。

輸入範例一

```
html_data = """
<div class="shop-wall">
    <div class="item">
        <a href="https://shop.com/prod/1">
            <img src="https://shop.com/img/thumb1.jpg">
        </a>
    </div>
    <div class="item">
        <a href="https://shop.com/prod/2">
            <img src="https://shop.com/img/thumb2.jpg">
        </a>
    </div>
</div>
"""
```

輸出結果一

商品連結：https://shop.com/prod/1 -> 圖片網址：https://shop.com/img/thumb1.jpg
商品連結：https://shop.com/prod/2 -> 圖片網址：https://shop.com/img/thumb2.jpg

實作題二：巢狀混雜文字與子標籤之完整提取（string 函數應用）

題目說明： 有些網頁的文章區塊內，文字中間會穿插大量的強烈提示標籤（如 <b>、 等）。如果使用 text() 會導致文字支離破碎且缺失子標籤內的文字。

1. 請解析指定的 HTML 字串。
2. 使用 XPath 的 string() 函數，直接提取出具有 class='article-box' 的 div 內部包含的所有文字（含子標籤文字）。
3. 使用 Python 的 .strip() 方法去除頭尾可能產生的換行與空白後印出結果。

提示：

- 使用 tree.xpath("string(//div[@class='article-box'])")。
- 注意：string() 函數在 XPath 中回傳的是一個**單一字串**，而不是列表，因此不需使用 [0] 取值。

輸入範例一

```
html_data = """
<div class="article-box">
    歡迎來到<b>Python 爬蟲特訓班</b>，本節課我們將深入探討<span>XPath</span>的強大進階功能！
</div>
"""
```

輸出結果一

歡迎來到 Python 爬蟲特訓班，本節課我們將深入探討 XPath 的強大進階功能！

9-3 XPath 常用函數：contains( ) 的強大應用

在實戰爬蟲中，網站的 ID、Class 常常是動態產生的，例如 post-12345、post-67890。這時 XPath 的 contains( ) 就能派上用場，只要部分符合條件即可找到目標。

一、觀念介紹

1. 模糊比對的必要性

- 應對動態屬性
  - 網站常使用 React、Vue 等框架動態產生 ID 或 Class。
  - 使用 contains( ) 可避免完全符合失效。
- 簡化複雜 Class
  - 標籤可能同時擁有多個 Class。
  - 只需鎖定其中具有代表性的關鍵字即可。

2. contains( ) 的語法結構

- [contains(@屬性, '字串')]：檢查屬性值是否包含指定文字。
- [contains(text( ), '字串')]：檢查標籤文字是否包含指定內容。

3. 邏輯組合

- 搭配 and、or、not( )：可建立更精準的篩選條件。

二、程式範例

範例 1. 抓取部分符合的 Class

```
from lxml import etree

html = """
<div class="container">
    <button class="btn btn-primary menu-item">首頁</button>
    <button class="btn btn-success menu-item">送出</button>
    <button class="btn btn-danger menu-item">刪除</button>
</div>
"""
tree = etree.HTML(html)

# 選取 class 中包含 'success' 的按鈕
success_btn = tree.xpath("//button[contains(@class, 'success')]/text( )")
print(f"找到的按鈕：{success_btn[0]}")
```

範例 2. 根據「顯示文字」定位標籤

```
from lxml import etree

html = """
<div class="user-profile">
    <p>姓名：張小明</p>
    <p>電話：0912-345-678</p>
    <p>電子郵件：ming@test.com</p>
</div>
"""
tree = etree.HTML(html)

# 抓取「包含」電話兩個字的 p 標籤文字
phone_info = tree.xpath("//p[contains(text( ), '電話')]/text( )")
print(f"抓取的資訊：{phone_info[0]}")
```

範例 3. 組合多個 contains 條件 (and / or / not)

```
from lxml import etree

html = """
<a href="/news/taiwan/1" class="link hot">台灣新聞</a>
<a href="/news/world/2" class="link">國際新聞</a>
<a href="/ad/click/3" class="link hot">贊助廣告</a>
"""
tree = etree.HTML(html)
# 條件：href 包含 'news' 且 class 包含 'hot'
news_links = tree.xpath("//a[contains(@href, 'news') and contains(@class, 'hot')]/text( )")
print(f"熱門新聞：{news_links}")

# 條件：class 包含 'link' 但「不包含」 'hot'
normal_links = tree.xpath("//a[contains(@class, 'link') and not(contains(@class,
'hot'))]/text( )")
print(f"一般連結：{normal_links}")
```

三、程式說明

1. contains( ) 的運作邏輯

- **布林判斷函數**：檢查第一個參數是否包含第二個參數。
- **屬性與文字寫法**：
  - **屬性**：@class
  - **文字**：text( )

2. text( ) vs .

- **contains(text( ), '字')**：只檢查目前標籤的文字。
- **contains(., '字')**：
  - 檢查目前標籤及所有子標籤文字。
  - 功能較強，但效能略低。

3. 運算子優先順序

- **建議加上括號：**混用 and、or 時可避免判斷錯誤。
- **例如**：//div[(contains(...) or contains(...)) and @id]

四、常見錯誤

錯誤 1：屬性漏掉 @ 符號

- **錯誤做法：**//div[contains(class, 'item')]
- **原因：**XPath 會把 class 當成子標籤名稱。
- **修正：**//div[contains(@class, 'item')]

錯誤 2：大小寫比對問題

- **原因：**XPath 1.0 的 contains( ) 區分大小寫。
- **範例：**contains(@class, 'main') 無法符合

錯誤 3：誤判文字節點

- **原因：**標籤內含換行或多個文字節點。
- **解法：**改用 contains(., '字串') 通常更穩定。

五、延伸重要觀念

1. 其他字串函數

- **starts-with(@attr, 'val')**：判斷屬性是否以指定字串開頭。
- **ends-with( )**：
  - XPath 1.0 不支援。
  - 通常改用 CSS Selector 或 Python 處理。

2. **萬用字元與 contains**
   - **任意標籤搜尋**：//*[contains(@id, 'post')]
     - 尋找所有 ID 包含 post 的元素。
3. **與 CSS Selector 的競爭力**
   - **CSS Selector**：擅長 Class、ID 選取。
   - **Xpath**：
     - 支援 contains(text( ), ...)。
     - 可直接根據畫面文字定位元素，是 XPath 的重要優勢。

9-3 XPath 常用函數：contains( ) 的強大應用 隨堂練習

一、選擇題 (共 10 題)

1. 當遇到網頁標籤的 ID 是動態生成的（例如 id="post-12345"，下次整理變 post-67890），最適合使用 XPath 的哪一個函數來進行模糊匹配？
   (A) starts-with( ) (B) ends-with( ) (C) contains( ) (D) matches( )
2. 在 XPath 中，若要選取 class 屬性中「包含」關鍵字 'success' 的所有 button 標籤，正確的語法結構為何？
   (A) //button[contains(class, 'success')]
   (B) //button[contains(@class, 'success')]
   (C) //button[@class='success']
   (D) //button[contains(text( ), 'success')]
3. 關於 contains(text( ), '字串') 與 contains(., '字串') 的細微差別，下列敘述何者正確？
   (A) 兩者完全沒有差別，在任何網頁結構下的效果都一樣
   (B) contains(text( ), ...) 會檢查當前標籤及其所有子標籤串接後的文字
   (C) contains(., ...) 僅檢查當前標籤的直接文字，不包含子標籤
   (D) contains(., ...) 會檢查當前標籤及其所有子標籤串接起來的文字，功能更強大
4. 小明想篩選出 href 包含 'news'「且」 class 包含 'hot' 的 a 標籤，他應該在 XPath 條件中使用哪一個邏輯運算子來連接？ (A) and (B) or (C) not (D) &&
5. 若要排除特定條件，例如選取 class 包含 'link' 但「不包含」 'hot' 的標籤，應該結合哪一個 XPath 函數？ (A) no( ) (B) except( ) (C) not( ) (D) ignore( )
6. 小明寫了 //div[contains(class, 'item')] 來尋找標籤，卻發現什麼都抓不到。請問他犯了什麼常見錯誤？
   (A) contains 只能用在 button 標籤上
   (B) 屬性名稱前面漏掉了 @ 符號，導致 XPath 誤以為在找名為 <class> 的子標籤
   (C) 應該把單引號 'item' 改成雙引號 "item"
   (D) 應該把 contains 改成 starts-with
7. 關於 XPath 1.0 的 contains( ) 函數對大小寫敏感度的描述，下列何者正確？
   (A) 完全不區分大小寫
   (B) 只有在搜尋 ID 屬性時才區分大小寫
   (C) 區分大小寫，若網頁原始碼寫 Class="Main"，搜尋 'main' 會宣告失敗
   (D) 會自動將所有網頁文字轉換為小寫後再行匹配
8. 除了 contains( ) 之外，XPath 還內建了哪一個可以用來檢查屬性是否以特定字串「開頭」的函數？ (A) starts-with( ) (B) ends-with( ) (C) begin-with( ) (D) head-with( )
9. 若想選取全網頁中「任何標籤」，只要該標籤的 ID 包含 'post' 即可，應該使用哪一個萬用字元配合 contains？

(A) //?[contains(@id, 'post')]　　(B) //all[contains(@id, 'post')]

(C) //.[contains(@id, 'post')]　　(D) //*[contains(@id, 'post')]

10. 關於 XPath 與 CSS Selector 的功能競爭力，下列哪項是 XPath 的 contains 函數目前在原生語法上仍「遙遙領先」CSS Selector 的強大功能？

(A) 根據標籤名稱進行定位 (B) 根據網頁上顯示的「文字內容」進行模糊匹配定位

(C) 根據 Class 的多個名稱進行精準匹配 (D) 根據 ID 進行快速選取

二、填充題 (共 5 題)

1. contains( ) 是一個_____函數 (回傳 True 或 False)，它會檢查第一個參數是否包含第二個參數。
2. 當網頁標籤內部含有換行符號或子標籤，導致 text( ) 回傳多個破碎片段時，改用 contains(_____, '字串') 通常能解決 90% 的定位問題。
3. 若要尋找一個按鈕，其上面顯示的文字包含「確認」，XPath 語法可寫為 //button[contains(_____, '確認')]。
4. 當在 XPath 中混用 and 與 or 等多重邏輯條件時，建議使用_____來確保運算的邏輯優先順序正確。
5. 常用的 XPath 1.0 版本中，內建支援 starts-with( )，但是並沒有內建可用於結尾匹配的 _____ 函數。

三、實作題 (共 2 題)

實作題一：個人資料欄位精準文字定位

題目說明： 請撰寫一段 Python 程式碼，使用 lxml 套件解析包含用戶個人隱私資料的 HTML 字串。

1. 許多網頁的欄位順序可能會變動，但欄位名稱不變。請使用 contains(text( ), '字串') 的技術，直接鎖定包含 '電話' 兩個字的 <p> 標籤。
2. 提取出該標籤的完整純文字內容。
3. 將提取出來的資訊印在螢幕上。

提示：

- 定位路徑可編寫為 //p[contains(text( ), '電話')]/text( )。
- 請記得從回傳的列表中使用 [0] 取出字串。

輸入範例一

```
html_data = """
<div class="bio">
    <p>會員帳號：user_01</p>
    <p>聯絡電話：0987-654-321</p>
    <p>通訊地址：台北市信義區</p>
```

```
</div>
"""
```

輸出結果一

```
提取結果：聯絡電話：0987-654-321
```

輸入範例二

```
html_data = """
<div class="bio">
    <p>住宅電話：02-22345678</p>
    <p>會員姓名：王大同</p>
</div>
"""
```

輸出結果二

```
提取結果：住宅電話：02-22345678
```

實作題二：動態多 Class 廣告標籤過濾（複合條件篩選）

題目說明： 某新聞網站的廣告與正常新聞混雜在 <a> 標籤中。正常新聞的 href 會包含 '/news/'；而廣告則會包含 '/ad/'。同時，熱門內容的 class 屬性會包含 'hot'。

1. 請編寫一條複合條件的 XPath，找出 href 包含 '/news/'「且」class「不包含」'hot' 的所有一般新聞 <a> 標籤的文字。
2. 使用迴圈依序印出過濾出來的一般新聞標題。

提示：

- 複合條件語法：//a[contains(@href, '/news/') and not(contains(@class, 'hot'))]/text()。

輸入範例一

```
html_data = """
<div class="feed">
    <a href="/news/politics/101" class="link hot-tag hot">焦點突發新聞</a>
    <a href="/news/sport/102" class="link item-style">平凡的體育賽事動態</a>
    <a href="/ad/products/5" class="link hot">超值商品限時下殺廣告</a>
    <a href="/news/tech/103" class="link item-style">科技新知每週報</a>
</div>
"""
```

輸出結果一

```
一般新聞：平凡的體育賽事動態
一般新聞：科技新知每週報
```

9-4 比較：什麼時候該用 BeautifulSoup？什麼時候該用 XPath？

BeautifulSoup 與 XPath 都能解析 HTML，但各有適合的使用情境。實務上通常會依網站特性與專案需求進行選擇。

一、綜合對比：兩大工具的特質分析

1. 核心設計理念的差異

- BeautifulSoup：
  - 容錯能力高。
  - 語法直覺、容易學習。
  - 適合快速開發。
- XPath (lxml)：
  - 查詢能力強。
  - 執行速度快。
  - 適合大型專案與複雜定位。

2. 特性對照表

特性	BeautifulSoup (CSS Selector)	XPath (lxml)
語法難度	簡單直覺	邏輯較強
解析速度	較慢	極快
容錯能力	極強	一般
查找方向	向下查找	全方位查找
內建功能	過濾功能豐富	函數與運算能力強

二、抉擇指南

何時使用 BeautifulSoup？

1. **面對結構混亂的網頁**
   - 可自動修補部分不完整 HTML。
   - 適合老舊網站。
2. **小型專案與快速開發**
   - 幾百頁資料量時已相當夠用。
   - 撰寫速度快。
3. **熟悉 CSS Selector**
   - 前端開發者上手最快。
   - 可直接使用 .select( )。

何時使用 XPath？

1. **效能需求高**：適合大量資料與大型 HTML。
2. **需要向上查找**：
   - 可找父節點、祖先節點與兄弟節點。
   - CSS Selector 較難做到。
3. **複雜篩選需求支援**：
   - contains( )
   - 屬性條件
   - 文字條件
   - 節點位置判斷

三、常見誤區

誤區 1：「XPath 一定比較快」

- **實際情況**：爬蟲最大的瓶頸通常是網路下載速度。
- **範例**：此時工具差異幾乎感受不到。
  - **網頁下載**：1 秒
  - **解析差異**：0.01 秒

誤區 2：兩者只能擇一

- **實務做法**：
  - BeautifulSoup 負責粗略解析。
  - XPath 負責精準定位。
- **混合使用**：
  - 先用 BeautifulSoup 抓區塊。
  - 再交給 lxml 使用 XPath 深入分析。

四、延伸重要觀念

1. 現代爬蟲框架的趨勢

- **Scrapy**：
  - Python 最知名爬蟲框架。
  - 預設大量使用 XPath。
- **Selenium / Playwright**：
  - 支援 CSS Selector 與 XPath。
  - XPath 在動態網頁定位時通常更有優勢。

2. 如何培養直覺？

- **初學階段**：
  - 優先使用 CSS Selector。
  - 建立 DOM 結構理解能力。
- **進階階段**：此時就是改用 XPath 的最佳時機。
  - 當 CSS Selector 開始變得冗長。
  - 或需要文字符合、父節點查找時。

9-4 比較：什麼時候該用 BeautifulSoup？什麼時候該用 XPath？ 隨堂練習

一、選擇題 (共 10 題)

1. 關於 BeautifulSoup 與 XPath (lxml) 的底層實作與效能差異，下列敘述何者正確？
   (A) BeautifulSoup 是由 C 語言實作，所以速度極快
   (B) lxml 是由底層 C 語言實作，處理大型文件時速度比 BeautifulSoup 快上許多
   (C) 兩者都是由 Java 實作，效能完全相同
   (D) XPath 因為功能較多，所以解析速度比 BeautifulSoup 慢很多
2. 哪一個工具因語法白話、對不對稱或結構混亂的「垃圾 HTML」程式碼容錯能力極強，而被定位為「給人類用的解析器」？ (A) Scrapy (B) Selenium (C) XPath (D) BeautifulSoup
3. 在網頁元素的「查找方向」上，XPath 擁有 CSS Selector 所沒有的哪項絕活？
   (A) 只能向下尋找子孫節點 (B) 完全無法在迴圈中使用
   (C) 可以全方位尋找，包含向上尋找父節點或兄弟節點
   (D) 只能尋找具有 ID 屬性的標籤
4. 當你需要進行簡單、小規模、需要快速開發出來的一次性爬蟲原型腳本時，優先選擇哪一個工具通常效率最高？
   (A) BeautifulSoup (B) Scrapy 框架 (C) XPath (D) 底層正則表達式
5. 如果你原本就是網頁前端工程師，已經對 CSS 選取器極為熟悉，在 BeautifulSoup 中可以使用哪一個方法來無縫接軌你的開發習慣？
   (A) find_all( ) (B) select( ) (C) xpath( ) (D) get_text( )
6. Python 最著名的重量級爬蟲框架 「Scrapy」，其內部預設優先推薦使用哪一種選取語法？ (A) BeautifulSoup (B) Regex (正則表達式) (C) XPath (D) JSONPath
7. 關於網頁爬蟲瓶頸的「迷思」，實際上限制爬蟲速度的最大瓶頸通常是什麼？
   (A) Python 迴圈執行的速度 (B) 網路延遲 (Latency) 與網頁下載時間
   (C) 電腦硬碟的寫入速度 (D) HTML 標籤的多寡
8. 關於 BeautifulSoup 與 XPath 的關係，下列哪一個觀念是正確的？
   (A) 它們是死對頭，在同一個專案內絕對不能共存
   (B) 兩者可以結合使用，例如先用 BeautifulSoup 抓大範圍，再轉字串交由 lxml 進行精細的 XPath 定位
   (C) 學會 XPath 之後，BeautifulSoup 就會被完全刪除無法使用
   (D) 只有自動化測試工具才能使用 XPath
9. 在自動化測試工具（如 Selenium 或 Playwright）中，面對動態生成的網頁元件或需要依賴「網頁顯示文字」來定位時，通常哪種語法支援度更深、更有威力？
   (A) CSS Selector (B) BeautifulSoup (C) XPath (D) Markdown
10. 職業級工程師在培養切換工具的「直覺」時，下列哪一個時機最適合從 CSS Selector 切換到 XPath？

(A) 當網頁只有一個簡單的 <h1> 標籤時

(B) 當發現標籤極難定位，或者 CSS 語法變得異常臃腫、需要逆向向上查找時

(C) 當需要將資料存入 CSV 檔案時

(D) 當網路速度變得非常慢的時候

二、填充題 (共 5 題)

1. BeautifulSoup 的核心設計理念是定位為「給_____用的解析器」，其優點是語法白話、容錯率高。
2. 許多老舊網站有開頭標籤卻沒結尾標籤，BeautifulSoup 內建的_____機制能自動處理這些髒資料，防止程式崩潰。
3. 當你想找到包含某段文字的表格，並選取該表格的父標籤時，CSS Selector 無法做到，但 XPath 可以做到，因為 XPath 支援_____向上的全方位查找。
4. 在網路爬蟲中，最大的效能瓶頸通常是 _____，而非解析套件本身的執行時間。
5. 知名 Python 爬蟲框架 Scrapy 預設優先推薦使用 XPath，最主要的原因就是為了其極佳的_____與擴充性。

三、實作題 (共 2 題)

實作題一：先大後小——混血爬蟲實戰（BS4 與 XPath 共存）

題目說明： 為了展示兩大工具並非死對頭，請撰寫一段 Python 程式碼，實現「共存流」爬取：

1. 傳入一段複雜的網頁 HTML。
2. **第一階段（使用 BeautifulSoup）**：先使用 BeautifulSoup 的 .select_one() 方法，精準切出核心資料區塊（.main-content）。
3. **第二階段（使用 XPath）**：將該核心區塊的內容轉為字串後，交由 lxml.etree.HTML() 解析，並直接使用一條 XPath 語法提取出內部所有 <li> 標籤的純文字。
4. 印出最終提取出的文字列表。

提示：

- 區塊轉字串可以使用 str(bs4_element)。
- 轉換後再使用 tree = etree.HTML(bs4_element_str)。
- 最後用 tree.xpath("//li/text()") 輕鬆抓出所有列表內容。

輸入範例一

```
html_data = """
<html>
<body>
    <nav class="top-nav">這裡是導覽列</nav>
    <div class="main-content">
```

```
        <h2>熱門課程清單</h2>
        <ul>
            <li>BeautifulSoup 基礎班</li>
            <li>XPath 進階特訓</li>
        </ul>
    </div>
    <footer class="footer">頁尾資訊</footer>
</body>
</html>
"""
```

輸出結果一

混血爬蟲抓取結果：['BeautifulSoup 基礎班', 'XPath 進階特訓']

實作題二：逆流而上——尋找目標標籤的父節點（XPath 獨門絕技）

題目說明： 網頁中有一個表格或區塊，我們只知道裡面有一段文字叫做 "關鍵密碼"，我們希望能抓到包覆這段文字的「最外層容器（具有 class='secret-box' 的 div）」裡面的 data-id 屬性。CSS Selector 沒辦法「往回/往上」抓，請用 XPath 的 ..（代表父節點）來完成它。

1. 解析指定的 HTML 字串。
2. 編寫 XPath 路徑：先用 contains(text(), '關鍵密碼') 定位到該 標籤。
3. 在路徑中使用 /.. 逆流向上退回到父節點 div。
4. 在路徑最後加上 /@data-id 直接提取出該屬性。
5. 印出提取到的 data-id 號碼。

提示：

- 逆流向上路徑示範：//span[contains(text(), '關鍵密碼')]/../@data-id。
- 不要忘記從回傳的列表中用 [0] 取得字串結果。

輸入範例一

```
html_data = """
<div class="secret-box" data-id="99988">
    <h3>機密檔案報告</h3>
    <p>這是一份未公開的內容。</p>
    <span>核心內容：關鍵密碼就在這裡</span>
</div>
"""
```

輸出結果一

成功逆流抓取父容器 ID：99988

CH9 綜合練習

一、選擇題 (共 25 題)

1. 關於 XPath 的定義與特性，下列敘述何者錯誤？
   (A) 它是一種專門用來在 XML 或 HTML 文件中尋找節點的語言
   (B) 它的定位邏輯像是透過「長相 (類別、ID)」來找人
   (C) 它不僅能向下尋找子節點，還能輕鬆地向上尋找父節點
   (D) 它是許多重量級爬蟲框架與自動化測試工具預設推薦的語法
2. 在 XPath 語法中，以一個斜槓 / 開頭的路徑稱為什麼？
   (A) 相對路徑 (B) 局部路徑 (C) 絕對路徑 (D) 萬用路徑
3. 為什麼在實際開發網頁爬蟲時，「不建議」大量使用絕對路徑？
   (A) 絕對路徑的執行速度比相對路徑慢數萬倍
   (B) 絕對路徑會被網頁的防火牆直接阻擋
   (C) 結構極其脆弱，網頁稍微改版或多一個 div，路徑就會直接報廢
   (D) 絕對路徑無法提取標籤內部的純文字
4. 在 XPath 語法中，以兩個斜槓 // 開頭的路徑稱為什麼？
   (A) 絕對路徑 (B) 相對路徑 (C) 鄰居路徑 (D) 父系路徑
5. 使用 lxml 套件時，核心方法 etree.HTML(html_str) 的主要功能是什麼？
   (A) 自動將 HTML 轉為 JSON 格式儲存
   (B) 補全不完整的 HTML 標籤並建立樹狀結構
   (C) 自動下載網頁內嵌的所有圖片
   (D) 執行網頁內含的 JavaScript 程式碼
6. 呼叫元素物件的 .xpath( ) 方法後，不論找到多少個符合條件的項目，其回傳值永遠是什麼型態？ (A) 字串 (String) (B) 字典 (Dictionary) (C) 元組 (Tuple) (D) 列表 (List)
7. 在 for 迴圈中遍歷多個容器時，若想從「當前節點」開始向下尋找東西，路徑開頭必須加上什麼符號？ (A) .. (兩個點) (B) . (一個點) (C) * (星號) (D) @ (老鼠號)
8. 小明在迴圈內誤寫了 item.xpath("//span") 而漏掉了點點，會導致什麼後果？
   (A) 程式直接拋出語法錯誤而中斷
   (B) 每一圈抓到的資料都變成「整份網頁的第一個 span」(全局選取陷阱)
   (C) 什麼東西都抓不到，一律回傳 None
   (D) 提取出來的文字會自動變成亂碼
9. 在 XPath 原生語法中，如果想要選取同層級的第二個 <li> 標籤，正確的寫法為何？
   (A) //li[2] (B) //li[1] (C) //li[0] (D) //li[-1]
10. 關於 XPath 與 CSS Selector 的路徑對應關係，下列何者正確？
    (A) XPath 的 //div/p 等同於 CSS 的 div p
    (B) XPath 的 //div//p 等同於 CSS 的 div > p
    (C) XPath 的 //div/p 等同於 CSS 的 div > p

(D) XPath 的 /html/body 等同於 CSS 的 html, body

11. 在 XPath 中，若要在路徑最後直接指定提取某節點下的「純文字內容」，應該使用哪一個函數？ (A) .text (B) string( ) (C) get_text( ) (D) text( )
12. 在 XPath 中，若要在路徑最後直接指定提取某標籤內部的屬性（如 href），必須在屬性名稱前加上什麼符號？ (A) # (B) . (C) @ (D) $
13. 若網頁結構為 ABC ，當執行 tree.xpath("//div/text( )") 時，回傳的結果是什麼？ (A) ['A', 'B', 'C'] (B) ['A', 'C'] (C) ['A B C'] (D) ['B']
14. 小明執行了 res = tree.xpath("//h1/text( )") 成功拿到資料，接著寫了 print(res[0].text) 卻報錯，原因為何？

    (A) res[0] 已經是 Python 字串物件，字串沒有 .text 屬性

    (B) h1 標籤必須先轉成 BeautifulSoup 物件

    (C) 必須寫成 res[0].text( ) 才能執行

    (D) xpath( ) 回傳的是布林值
15. 如果想抓取某標籤內「包含所有子標籤內部文字」的完整串接內容（類似 BeautifulSoup 的 .text），在 XPath 中應改用什麼函數？

    (A) all_text( ) (B) string( ) (C) text(all) (D) content( )
16. 當網頁標籤的 ID 或 Class 是動態生成、帶有隨機數值時，最適合使用哪一個 XPath 函數來進行模糊匹配？ (A) starts-with( ) (B) contains( ) (C) ends-with( ) (D) matches( )
17. 下列哪一個 XPath 語法代表選取 class 屬性中「包含 hot」的所有 a 標籤？

    (A) //a[contains(class, 'hot')] (B) //a[@class='hot']

    (C) //a[contains(@class, 'hot')] (D) //a[contains(text( ), 'hot')]
18. 在 XPath 中，多個模糊匹配條件如果想要同時滿足（即「且」的邏輯），應該使用哪一個運算子連接？ (A) and (B) or (C) not (D) &&
19. 下列哪一個 XPath 語法代表選取「任何標籤」，只要該標籤的 ID 屬性中包含 'post' 即可？

    (A) //.[contains(@id, 'post')] (B) //div[contains(@id, 'post')]

    (C) //?[contains(@id, 'post')] (D) //*[contains(@id, 'post')]
20. 根據課程所述，常用的 XPath 1.0 標準中，內建支援了 starts-with( )，但「沒有」內建哪一個結尾匹配函數？

    (A) ends-with( ) (B) contains-with( ) (C) finish-with( ) (D) tail-with( )
21. 關於 BeautifulSoup 與 XPath (lxml) 的核心設計理念，下列敘述何者正確？

    (A) BeautifulSoup 定位為高性能的文件查詢語言

    (B) XPath 定位為給人類用的解析器，容錯度極高

    (C) BeautifulSoup 語法直覺白話，目標是讓開發者用最少的腦力成本抓到資料

    (D) lxml 是專門為小規模、一次性開發設計的

22. 面對老舊網站中大量標籤不對稱、結構極其混亂的「垃圾 HTML」時，優先選擇哪一個工具最具優勢？ (A) XPath (B) BeautifulSoup (C) 正則表達式 (D) Scrapy 內建定位器
23. 在進行大型、海量數據（如數萬筆網頁文件）的爬取與解析時，為什麼頂尖工程師會優先切換到 XPath (lxml)？
    (A) 因為 lxml 的底層是用 C 語言實作，解析速度極快
    (B) 因為 XPath 可以自動繞過網頁的反爬蟲機制
    (C) 因為 XPath 寫起來比 CSS Selector 更加簡短白話
    (D) 因為 lxml 可以自動把資料存入資料庫
24. 在網路爬蟲的實戰效能中，限制整體爬取速度的最大瓶頸通常是什麼？
    (A) 解析套件（如 BS4 或 lxml）的執行時間
    (B) 網路延遲（Latency）與網頁下載時間
    (C) 電腦 CPU 的核心數量
    (D) Python 編輯器的編譯速度
25. 關於兩大解析工具的關係，下列哪一個觀念是職業級工程師應具備的？
    (A) 它們是死對頭，在同一個 Python 檔案內絕對不能並存
    (B) 學會 XPath 之後，就應該完全拋棄並忘記 BeautifulSoup
    (C) 它們可以共存，例如先用 BeautifulSoup 抓取大範圍區塊，再用 lxml 進行精細的 XPath 定位
    (D) 自動化測試工具如 Selenium 只能支援 CSS Selector，不支援 XPath

二、填充題 (共 15 題)

1. XPath 就像是透過「_____（路徑）」來找網頁元素，而 CSS Selector 則是透過「長相」來找。
2. 絕對路徑是以一個斜槓 / 開頭，而相對路徑則是以_____開頭。
3. 在進行容器化局部查找時，路徑中的點雙斜槓 .// 代表「從_____節點開始找」。
4. 由於 XPath 語法中的索引值是從 _____ 開始計算，這與 Python 清單從 0 開始不同，需要特別注意。
5. 函數 text() 只會抓取當前層級的純文字，若要穿透並包含所有子標籤內部的文字，必須改用 _____ 函數。
6. 若要在 XPath 中提取標籤的屬性值，必須在屬性名稱前加上 _____ 符號。
7. 當 tree.xpath() 在整份網頁中找不到任何匹配的項目時，預設會回傳一個_____給 Python。
8. 為了將全域同時提取出來的「標題字串列表」與「網址字串列表」在 for 迴圈中一一對應縫合，可以呼叫 Python 內建的 _____ 函數。
9. contains() 函數是一個_____函數，它會檢查第一個參數是否包含第二個參數。
10. 若要選取 class 包含 'link' 但不包含 'hot' 的 a 標籤，可寫成 //a[contains(@class, 'link') and _____]/text()。

11. 很多現代網站使用框架生成帶有隨機後綴的 ID，這種屬性在觀念中被稱為_____屬性。
12. lxml 處理超大型網頁的速度非常快，主要是因為它的底層是用 _____ 語言編寫的。
13. Python 最著名的重量級高級爬蟲框架 _____，其內部預設優先推薦使用 XPath。
14. 在 Chrome 瀏覽器的開發者面板（F12）中，按下快速鍵 _____ 即可開啟搜尋框，即時輸入並測試 XPath 語法。
15. 在 XPath 中如果要表達「逆流向上尋找當前標籤的父節點」，可以使用 _____ 符號。

三、實作題 (共 3 題)

實作題一：成員卡片之局部路徑防禦性提取（結合 .// 與缺失值處理）

題目說明： 請撰寫一段 Python 程式碼，使用 lxml 套件解析含有多個員工卡片的 HTML 字串。

1. 使用相對路徑先找出所有的員工容器（具有 class='emp-card' 的 div）。
2. 使用 for 迴圈遍歷每個容器。
3. 在迴圈內部，務必使用局部相對路徑（帶有點點 .//）分別提取員工的姓名（class='name'）與分機號碼（class='ext'）。
4. **防錯機制**：某些新進員工可能尚未分配到分機號碼（缺少該 span 標籤），若找不到分機資料，請預設顯示 "暫無分機"。
5. 依序將每位員工的資料格式化印出。

提示 1：容器選取路徑：//div[@class='emp-card']。

提示 2：迴圈內部使用 card.xpath(".//span[@class='name']/text()") 提取姓名。

提示 3：針對分機，先用 card.xpath(".//span[@class='ext']/text()") 取出列表，再判斷該列表是否為空。若不為空則取 [0]，若為空則賦予 "暫無分機"。

輸入範例一

```
html_str = """
<div class="company-directory">
    <div class="emp-card">
        <span class="name">艾莉絲</span>
        <span class="ext">分機 101</span>
    </div>
    <div class="emp-card">
        <span class="name">鮑伯</span>
        </div>
</div>
"""
```

輸出結果一

員工：艾莉絲 | 分機：分機 101

員工：鮑伯 | 分機：暫無分機

實作題二：動態屬性與複合條件過濾（contains 與 not 綜合應用）

題目說明： 某論壇網頁的文章列表結構如下，其中包夾了正常的討論貼文與贊助商廣告。貼文的 id 是動態生成的（例如 id="post-99"），且熱門貼文的 class 會包含 'hot'。

1. 請編寫一條複合條件的 XPath，直接全域提取出所有符合以下條件的文章標題文字：
    - id 屬性中必須**包含** 'post-'。
    - class 屬性中**不能包含** 'hot'（即只要過濾出一般普通貼文）。
2. 使用迴圈依序將過濾出來的一般貼文標題印出。

提示：

- 本題要求直接從全域提取出字串列表，不需要先抓取容器。
- 複合條件語法結構可參考：//h3[contains(@id, '...') and not(contains(@class, '...'))]/text()。

輸入範例一

```
html_str = """
<div class="forum-list">
    <h3 id="post-201" class="title hot-topic hot">【焦點】2026 年最新科技趨勢預測</h3>
    <h3 id="post-202" class="title normal-topic">【問題】請問 Python 爬蟲推薦用哪種套件？</h3>
    <h3 id="ad-999" class="title hot">【贊助】限時英文線上課程特價中</h3>
    <h3 id="post-203" class="title normal-topic">【心得】轉職後端工程師一年的心路歷程</h3>
</div>
"""
```

輸出結果一

一般貼文：【問題】請問 Python 爬蟲推薦用哪種套件？

一般貼文：【心得】轉職後端工程師一年的心路歷程

實作題三：逆流向上尋找父節點容器（XPath 獨門絕技）

題目說明： 在爬取複雜的商品表格或卡片時，我們往往只知道某個特定的內文（例如特定的商品編號 "SN-999"），但我們真正想抓取的，是包覆著這個編號的「最外層商品容器（div）」上面的動態資料屬性 data-price（價格）。由於 CSS Selector 無法往回、向上查找，請利用 XPath 的 .. 語法完成此任務：

1. 解析指定的 HTML 字串。
2. 編寫一條 XPath 路徑：先利用 contains(text(), 'SN-999') 精準定位到顯示商品編號的 標籤。
3. 在路徑中加上 /.. 逆流向上退回到其父節點（即具有 class='product-box' 的 div）。
4. 在路徑最後加上 /@data-price 直接提取出該父容器的價格屬性。
5. 印出提取到的商品價格。

提示：

- 逆流向上的完整路徑設計：//span[contains(text(), 'SN-999')]/../@data-price。
- 記得從 .xpath() 回傳的列表中使用 [0] 取得字串結果。

輸入範例一

```
html_str = """
<div class="product-box" data-price="3500">
    <h4 class="p-name">高音質無線耳機</h4>
    <p>原廠保固一年，附贈收納盒。</p>
    <span class="p-sn">序號：SN-999</span>
</div>
<div class="product-box" data-price="1200">
    <h4 class="p-name">人體工學滑鼠</h4>
    <p>極致舒適手感，辦公必備。</p>
    <span class="p-sn">序號：SN-888</span>
</div>
"""
```

輸出結果一

```
查找到的商品價格為：3500
```